

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I P Dinesh Reddy (192525399) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.

Reverse Engineering of a Big Data Website and Customer Analytics Platform

1. Probable Distributed Storage System

A distributed storage system such as **HDFS or cloud object storage** is likely used to store billions of click, mobile, and transaction records. Data is distributed across multiple nodes for scalability and fault tolerance.

2. Data Ingestion Framework

Tools such as **Apache Kafka, Flume, or cloud-based ingestion services** likely collect continuous website clicks, mobile interactions, and customer transactions.

3. Data Organization and Indexing

- **Structured data:** Transaction records are stored in structured databases/data warehouses.
- **Semi-structured data:** Clickstreams and mobile logs are stored in formats such as JSON/Parquet.
- Partitioning by **date, region, or customer** improves query performance.

4. Batch and Stream Processing

Apache Spark is likely used for large-scale batch processing, while **Kafka Streams or Spark Streaming** supports real-time analytics and reporting.

5. Scalability and Partitioning

The platform uses **horizontal scaling**, data partitioning, replication, and parallel processing. Additional nodes can be added as data volume increases.

6. Analytics and BI Layer

Processed data is transferred to an **analytics/data warehouse layer**, where BI tools and dashboards generate reports, KPIs, customer trends, and business insights.

Probable Architecture

Data Sources → Kafka/Ingestion → Distributed Storage → Spark Processing → Analytics/Data Warehouse → BI Dashboards

Design Decisions

The architecture is designed for **high-volume data, fast ingestion, parallel processing, fault tolerance, scalability, and real-time business reporting**. These decisions directly address the requirement to process billions of records daily. The task specifically requires inference of storage, ingestion, processing, indexing, streaming, scalability, and BI connectivity.

2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.

Reverse Engineering of an E-Commerce Big Data Platform

1. Event Streaming

Tools such as **Apache Kafka** are likely used to capture cart additions, product views, abandoned checkouts, and payment events continuously across regions.

2. Customer Profiling and Recommendation

A **distributed customer database** stores customer profiles and behavioral data. Machine-learning models analyze browsing and purchase history to generate personalized product recommendations.

3. Data Synchronization

Session data, clickstream logs, and transaction records are synchronized through the **event-streaming pipeline**, allowing real-time personalization based on current customer activity.

4. Caching and Query Processing

A distributed cache such as **Redis** can provide fast access to frequently used customer and product information. A distributed query engine processes large datasets efficiently.

5. Machine Learning Pipeline

The workflow includes **data collection** → **preprocessing** → **feature extraction** → **model training** → **recommendation generation**, enabling real-time personalized recommendations.

6. Data Integration

Structured **order and payment data** is combined with semi-structured **browsing and clickstream data** through distributed processing and transformation pipelines.

7. Scalability and Fault Tolerance

The system uses **partitioning, replication, horizontal scaling, and distributed processing** to handle increasing users and regional workloads while maintaining availability during failures.

8. Dashboard and KPI Monitoring

Processed data feeds dashboards that monitor **sales, conversion rate, cart abandonment, customer activity, and recommendation performance**.

Probable Architecture

Customer Events → **Kafka** → **Distributed Storage** → **Processing/Query Engine** → **ML Recommendation** → **Cache/Analytics** → **KPI Dashboards**

Design Decision

The architecture prioritizes **real-time event processing, personalization, low-latency access, scalability, and fault tolerance**. These components directly correspond to the task's requirements for event streaming, customer profiling, synchronization, caching, distributed querying, machine learning, and KPI monitoring.

3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

Reverse Engineering of a Big Data Healthcare System

1. Hybrid Database Architecture

A **hybrid storage architecture** is likely used to manage structured patient histories and prescription records along with unstructured diagnostic images. Structured data can be stored in relational databases, while images and other large files can use distributed storage.

2. Data Integration

Data integration tools connect **hospitals, laboratories, and insurance systems**, enabling patient information to be collected and combined from multiple sources.

3. Streaming and Machine Learning

Streaming frameworks process **wearable sensor data continuously**, while machine-learning frameworks analyze patient information for **disease prediction and health monitoring**.

4. Privacy and Security

The platform likely applies **encryption, access control, authentication, auditing, and compliance mechanisms** to protect sensitive healthcare information.

5. Analytics Pipeline

Data Collection → Integration → Secure Storage → Processing → Machine Learning → Clinical Analytics

The processed information supports **clinical decision-making and population health studies**.

6. Probable Architecture

Hospitals + Laboratories + Insurance + Wearables → Data Integration → Hybrid Storage → Streaming/ML Processing → Healthcare Analytics

7. Design Decisions

The architecture is designed to handle **structured, unstructured, and streaming healthcare data** while maintaining privacy, security, and regulatory compliance. It supports both **real-time monitoring** and large-scale healthcare analytics. These requirements are explicitly identified in the task.

4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

Reverse Engineering of a Smart City Big Data Platform

1. Edge Computing Setup

Traffic sensors, CCTV systems, weather sensors, and public transport devices likely use **edge computing** to collect and preprocess data close to its source, reducing latency.

2. Distributed Messaging and Storage

A distributed messaging system such as **Apache Kafka** can continuously collect sensor feeds. Centralized distributed storage stores large volumes of collected city data.

3. Geospatial and Streaming Analytics

Geospatial analytics processes location-based information, while streaming computations analyze traffic and transport feeds in real time to predict congestion.

4. Databases

Time-series databases store continuously generated sensor measurements, while **location-aware databases** support geographic and spatial queries.

5. Batch and Real-Time Analytics

Real-time analytics provides immediate operational insights, while batch analytics analyzes historical data for **urban planning and long-term decision-making**.

6. Security and Access Control

The platform likely uses **authentication, authorization, encryption, and role-based access control** to protect citizen and infrastructure data.

7. Visualization

Visualization systems and dashboards present **live traffic conditions, congestion predictions, weather information, and public transport activity** for operational monitoring.

Probable Architecture

Sensors/CCTV/Transport → Edge Computing → Distributed Messaging → Centralized Storage → Streaming + Batch Analytics → Geospatial Analytics → Live Dashboards

Design Decisions

The architecture emphasizes **low-latency processing, continuous data ingestion, scalable storage, geospatial analysis, security, and real-time visualization**, matching the requirements described in the task.

5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering of a Healthcare Analytics System

1. Hybrid Storage Architecture

A **hybrid storage architecture** is likely used to manage structured patient records, laboratory reports, and appointment histories along with unstructured medical data and wearable readings.

2. ETL and Interoperability

ETL pipelines extract, transform, and load data from hospitals and clinics. Healthcare interoperability standards are likely used to exchange and integrate patient information between systems.

3. Real-Time Monitoring

Wearable device readings can be processed through a **real-time streaming pipeline** to continuously monitor patient conditions and identify abnormal patterns.

4. Predictive Health Analytics

Machine-learning models analyze patient records, laboratory results, wearable data, and appointment histories to support **diagnosis risk scoring and treatment recommendations**.

5. Distributed Processing

Distributed processing frameworks enable large-scale analysis of patient data for **population-level healthcare studies**, providing scalable and parallel computation.

6. Security and Governance

Sensitive medical data is protected using **encryption, access control, compliance mechanisms, and audit trails** to maintain data security and accountability.

7. Probable Architecture

Patient Records + Wearables + Labs + Appointments → ETL/Interoperability → Hybrid Storage → Distributed/Real-Time Processing → ML Analytics → Risk Scoring & Treatment Recommendations

8. Design Decisions

The system is designed for **secure data integration, real-time monitoring, scalable population analytics, and predictive healthcare support**. These design requirements directly match the reverse-engineering criteria given in the task.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented